

Nicholas Carter

policy-gated AI systems, async data flows, and reproducible evidence

Sarasota, Florida · Open to senior/staff engineering roles and selected client work

mr.nicholas.b.carter@gmail.com · GitHub · LinkedIn

Summary

I build policy-gated AI infrastructure, evidence-bound software automation, and quantitative risk systems. My work favors deterministic controls, explicit failure modes, and proof that can be reproduced.

Selected public work

Verdict · [AI control plane](#)

A fail-closed control plane that filters models through deterministic capability, privacy, reliability, and policy gates before advisory ranking.

Boundary: The public credential-free path uses checked-in fixtures and simulated failure. It proves routing behavior and receipts—not live-provider availability, adoption, or production scale.

Python · [policy gates](#) · [receipts](#) · [failover](#)

Prediction Market SDK · [Async market infrastructure](#)

A typed Python foundation for Kalshi REST workflows, reusable L2 order-book state, and resilient WebSocket lifecycle management.

Boundary: Kalshi signing, order workflows, order-book snapshots and deltas, and subscription management are implemented and tested. Polymarket signing and live-order safety remain limited.

Python · [AsyncIO](#) · [WebSockets](#) · [msgspec](#)

Verdict Risk · [Deterministic risk](#)

Pure-functional capital protection primitives that separate risk decisions from persistence and network I/O.

Boundary: The repository exposes deterministic drawdown, position, cluster exposure, and Kelly-sizing gates with property-based tests.

Python · [risk gates](#) · [Hypothesis](#) · [OpenTelemetry](#)

Capabilities

AI infrastructure

- Policy-gated model routing
- Fail-closed authorization
- Agent orchestration
- MCP integration
- Evidence receipts

Backend & data

- Python
- AsyncIO
- FastAPI
- WebSockets
- SQLite
- Typed contracts

Platform engineering

- CI/CD
- Contract parity
- Security gates
- Failure recovery
- Reproducible testing

Evidence note: This resume is deliberately limited to public, repository-verifiable project work. No unverified employment timeline or performance metric is included.